

Министерство образования Республики Беларусь

Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

Лабораторная работа №2

по дисциплине «Проектирование защищенных интеллектуальных
информационных систем»

на тему «Анализ защищённости приложений.
Часть 1. Оценка защищённости разработанного приложения»

Выполнил студент гр. 321701

Ковальчук В. Н.
Самович В. М.

Проверил

Сальников Д. А.

Минск 2026

Цель: оценить защищённость самостоятельно разработанного приложения – проанализировать политику безопасности операционной системы, компонентов приложения и их взаимодействия, сетевую политику безопасности и защищённость конфиденциальной информации – и получить практические навыки обоснованной защиты пользовательских данных на уровне архитектуры приложения, а не только на уровне ОС или СУБД.

Задание:

- а) Разработать приложение, которое позволяет:
 - добавить пользователя в систему;
 - авторизовать пользователя на основе идентификационных данных;
 - создать, редактировать, удалить, осуществить поиск конфиденциальных данных;
 - создать, редактировать, удалить, осуществить поиск неконфиденциальных данных;
 - деавторизовать авторизованного пользователя.
- б) Выполнить анализ разработанного приложения с точки зрения: политики безопасности операционной системы; политики безопасности компонентов приложения и их взаимодействия между собой; сетевой политики безопасности; безопасности конфиденциальности данных.
- в) Оформить отчёт о проделанной работе.

Постановка задачи. Приложение **SecureApp** реализовано как типовое клиент-серверное веб-приложение: SPA-клиент на React обращается по HTTP(S) к REST API на ASP.NET Core, который хранит данные в локальной базе SQLite. Приложение ведёт учёт пользователей и двух типов пользовательских записей – конфиденциальных и неконфиденциальных – в объёме, достаточном для содержательного анализа всех четырёх аспектов защищённости, перечисленных в задании, без привязки к какой-либо предметной области.

Краткое описание разработанного приложения

SecureApp – клиент-серверное веб-приложение, состоящее из двух независимо развёртываемых частей.

– **Backend/SecureApp.Api** – REST API на ASP.NET Core (.NET), хранящее данные в локальной базе SQLite через Entity Framework Core. Реализует все операции задания: регистрацию (POST /api/auth/register), вход (POST /api/auth/login), выход (POST /api/auth/logout) и по четыре операции CRUD и поиска для каждого из двух типов данных (/api/confidential, /api/public).

– **Frontend/secure-app-ui** – одностраничное приложение на React (TypeScript) с компонентами Material UI: формы входа и регистрации, а также панель с двумя вкладками – «Confidential data» и «Public data», – в каждой из которых таблица записей, строка поиска и диалог создания/редактирования.

Каждая запись (конфиденциальная и неконфиденциальная) принадлежит ровно одному пользователю: сервер связывает её с **OwnerId** из проверенного токена сессии, а не с значением, присланным клиентом, поэтому чтение, изменение, удаление и поиск всегда выполняются в границах данных текущего пользователя.

Анализ защищённости приложения

1. Политика безопасности операционной системы

Приложение не хранит секреты в коде и не рассчитывает на права операционной системы для защиты данных – оно рассчитывает на собственный уровень контроля доступа (см. далее), что делает его защищённость переносимой между ОС. Тем не менее ОС-уровень остаётся значимым: ключ подписи JWT и ключ шифрования конфиденциальных данных вынесены из кода в конфигурацию (`appsettings.Development.json` – только для локальной разработки) с явным указанием, что в эксплуатации они должны задаваться переменными окружения (`Jwt__Key`, `Encryption__Key`) или менеджером секретов, а не файлом, попадающим в систему контроля версий; в этом случае получить их может только тот, у кого есть права на переменные окружения процесса, то есть тот же пользователь ОС, что и владелец процесса сервера. Файл базы данных SQLite создаётся с правами по умолчанию текущего пользователя ОС (`umask` процесса), то есть недоступен на чтение другим локальным пользователям без дополнительных привилегий.

2. Политика безопасности компонентов приложения и их взаимодействия

Взаимодействие «клиент – сервер» построено на нескольких независимых рубежах защиты, которые не дублируют, а дополняют друг друга.

Во-первых, аутентификация не хранит пароль – при регистрации пароль преобразуется алгоритмом PBKDF2-HMACSHA256 со случайной солью на пользователя и 210 000 итераций (см. `PasswordHasher.cs`), после чего сравнивается только результат хеширования. Во-вторых, состояние сессии передаётся не в теле ответа и не в заголовке, а в cookie-заголовке с атрибутами `HttpOnly` (недоступен JavaScript-коду страницы – защита от кражи токена через XSS), `Secure` (передаётся только по защищённому соединению) и `SameSite=Strict` (не отправляется при переходах с посторонних сайтов – защита от CSRF), что и подтверждено в листинг 3 (Шаг 3). В-третьих, каждый защищённый маршрут API отмечен атрибутом `[Authorize]` и дополнительно фильтрует данные по идентификатору владельца, извлечённому из подписанного токена, а не переданному клиентом явно (см. листинг 1), поэтому компонент хранения данных и компонент разграничения доступа физически не могут разойтись – условие владения встроено в каждый запрос к базе.

```

1 private int CurrentUserId =>
  ↪ int.Parse(User.FindFirstValue(ClaimTypes.NameIdentifier!));
2
3 [HttpGet("{id:int}")]
4 public async Task<ActionResult<RecordResponse>> Get(int id)
5 {
6     var record = await db.ConfidentialRecords
7         .SingleOrDefaultAsync(r => r.Id == id && r.OwnerId ==
8         ↪ CurrentUserId);
9     return record is null ? NotFound() : Ok(ToResponse(record));
10 }

```

Листинг 1 – Каждый запрос к конфиденциальным данным ограничен владельцем записи

Обращение постороннего пользователя к чужой записи намеренно возвращает 404 Not Found, а не 403 Forbidden: с точки зрения пользователя, не владеющего записью, между «записи не существует» и «записи существует, но доступ запрещён» не должно быть заметной разницы – иначе сам код ответа стал бы каналом утечки информации о существовании чужих идентификаторов.

Наконец, попытки входа с неверным паролем не различаются с попытками входа под несуществующим именем пользователя – в обоих случаях возвращается одно и то же сообщение (листинг 3, Шаг 4), что не позволяет использовать форму входа для перебора существующих в системе имён пользователей.

3. Сетевая политика безопасности

Взаимодействие фронтенда и API ограничено списком разрешённых источников (CORS), настроенным по одному конкретному адресу фронтенда, а не по маске или символу *, причём заголовок Access-Control-Allow-Credentials выставляется только для этого источника – запрос с постороннего источника не получает в ответе заголовок Access-Control-Allow-Origin вовсе (листинг 3, Шаг 10), то есть браузер блокирует его на своей стороне ещё до того, как ответ окажется доступен вызывающему скрипту. Дополнительно ко всем ответам добавляются заголовки X-Content-Type-Options: nosniff, X-Frame-Options: DENY и Referrer-Policy: no-referrer (см. Program.cs), снижающие, соответственно, риск MIME-sniffing атак, встраивания страницы в чужой <iframe> (clickjacking) и утечки внутренних путей приложения через заголовок Referer при переходе по внешним ссылкам. Транспорт защищён на уровне промежуточного ПО UseHttpsRedirection, форсирующего HTTPS в развёрнутой конфигурации. Наконец, на сетевом канале входа реализовано

вана защита от перебора паролей: после пяти неудачных попыток входа под одним и тем же именем дальнейшие попытки отклоняются кодом **429 Too Many Requests** на 30 секунд вне зависимости от корректности пароля – продемонстрировано в листинг 3 (Шаг 5), где шестая попытка отклонена, хотя пароль в ней уже верен.

Анализ защищённости конфиденциальной информации

Различие между конфиденциальными и неконфиденциальными данными в **SecureApp** проведено не только на уровне доступа (см. выше), но и на уровне хранения. Неконфиденциальные записи хранятся в столбцах типа **TEXT** открытым текстом. Конфиденциальные записи перед записью в базу шифруются алгоритмом AES-256-GCM с ключом, производным от конфигурационного секрета (листинг 2): используется аутентифицированное шифрование, то есть попытка подменить зашифрованные байты в базе напрямую (в обход API) приведёт не к подстановке произвольных данных, а к ошибке проверки тега аутентификации при расшифровке.

```
1 public byte[] Encrypt(string plaintext)
2 {
3     var nonce = RandomNumberGenerator.GetBytes(NonceSize);
4     var plainBytes = Encoding.UTF8.GetBytes(plaintext);
5     var cipherBytes = new byte[plainBytes.Length];
6     var tag = new byte[TagSize];
7
8     using var aes = new AesGcm(_key, TagSize);
9     aes.Encrypt(nonce, plainBytes, cipherBytes, tag);
10    // Результат: [nonce][ciphertext][tag] -- всё, что попадает в столбец BLOB.
11    ...
12 }
```

Листинг 2 – Шифрование конфиденциального поля перед сохранением (AES-256-GCM)

Утверждение о шифровании проверено не по коду, а экспериментально: в базу была записана конфиденциальная запись со значением 4111-0000-0000-0000 и неконфиденциальная запись со значением **Office closed Friday**, после чего необработанный файл базы данных (включая WAL-журнал SQLite) был просканирован утилитой **strings**. Конфиденциальное значение не встретилось в файле ни в каком виде, тогда как неконфиденциальное было найдено открытым текстом (листинг 3, Шаг 11) – то есть даже полная утечка файла базы данных (например, копированием диска) не раскрывает конфиденциальные поля без отдельно хранимого ключа шифрования, тогда как неконфиденциальные поля таким сценарием раскрываются полностью, что ожидаемо и соответствует их статусу.

Полный протокол демонстрации, включающий регистрацию, вход, разграничение данных между двумя пользователями, поиск, CORS и завершение сессии, приведён ниже.

```

1  == Step 1: Register two independent users ==
2  [OK] Register 'alice' $ curl -X POST http://localhost:5080/api/auth/register -d '{"username":"alice",...}'
3  [OK] Register 'bob'
4  == Step 2: Registration input validation ==
5  [OK] Reject username/password shorter than the minimum length (HTTP 400)
6  [OK] Reject duplicate username 'alice' (HTTP 409)
7  == Step 3: Log in both users and inspect the session cookie ==
8  [OK] Log in as 'alice' (HTTP 200)
9  [OK] Session cookie carries HttpOnly; Secure; SameSite=Strict
10 [OK] Log in as 'bob'
11 == Step 4: Wrong password and unknown username return the identical response ==
12 [OK] Wrong password and unknown username both return: {"message":"Invalid username or password."}
13 == Step 5: Brute-force lockout after repeated failed logins ==
14 [OK] 6th login for 'bruteuser' rejected with HTTP 429 even though the password is now correct
15 == Step 6: Unauthenticated access to protected endpoints ==
16 [DENIED ] Read          GET /api/confidential (no session) (HTTP 401)
17 [DENIED ] Read          GET /api/public (no session) (HTTP 401)
18 == Step 7: 'alice' creates one confidential and one public record ==
19 [OK] Create confidential record #1 (title='Card')
20 [OK] Create public record #1 (title='Notice')
21 == Step 8: Owner can read, search, update and delete their own records ==
22 [ALLOWED] Read          GET confidential/1 (owner) (HTTP 200)
23 [ALLOWED] Search        confidential?query=card (owner) (HTTP 200)
24 [ALLOWED] Update        PUT confidential/1 (owner) (HTTP 200)
25 == Step 9: A second user cannot see, edit or delete the first user's records ==
26 [DENIED ] Read          GET alice's confidential/1 (bob) (HTTP 404)
27 [DENIED ] Update        PUT alice's confidential/1 (bob) (HTTP 404)
28 [DENIED ] Delete        DELETE alice's confidential/1 (bob) (HTTP 404)
29 [OK] 'bob' lists confidential records: sees none of alice's data ([])
30 == Step 10: Cross-origin requests from an unregistered origin are rejected ==
31 [OK] OPTIONS preflight from http://evil.example carries no Access-Control-Allow-Origin header
32 [OK] OPTIONS preflight from the registered frontend origin (http://localhost:5175) is allowed
33 == Step 11: Confidential data is unreadable in the raw database file, public data is not ==
34 [OK] Confidential content '4111-0000-0000-0000' not found anywhere in the raw database file
35 [OK] Public content 'Office closed Friday' IS found in plain text (expected: non-confidential)
36 == Step 12: Logging out invalidates the session ==
37 [OK] GET /api/auth/me after logout is rejected (HTTP 401)
38 === Summary ===
39 Operations: 25 total, 0 failed
40   Delete      allowed 0 / 1   denied 1
41   Read        allowed 1 / 4   denied 3
42   Search      allowed 1 / 1   denied 0
43   Update      allowed 1 / 2   denied 1

```

Листинг 3 – Реальный протокол проверки SecureApp (скрипт verify.sh)

Выводы

В ходе выполнения лабораторной работы было разработано и практически проверено веб-приложение, реализующее полный набор операций из задания (регистрация, авторизация, CRUD и поиск для конфиденциальных и неконфиденциальных данных, деавторизация), а также выполнен анализ его защищённости по всем четырём требуемым направлениям: политике безопасности операционной системы, политике безопасности компонентов приложения и их взаимодействия, сетевой политике безопасности и защищённости конфиденциальной информации. В отличие от первой лабораторной работы, где контроль доступа обеспечивался средствами ОС и СУБД, здесь все рубежи защиты – от хеширования пароля до шифрования конфиденциального поля – реализованы и обоснованы на уровне кода самого приложения, что и являлось целью работы.

Семантически работа показала, что защищённость приложения складывается из нескольких независимых, не подменяющих друг друга механизмов: то, что пароль нельзя восстановить из базы (хеширование), не отменяет необходимости защищать сам токен сессии от кражи (**HttpOnly**, **Secure**, **SameSite**), а то, что токен защищён, не отменяет необходимости проверять владельца записи на каждом запросе к данным – все три меры проверялись по отдельности и ни одна не компенсирует отсутствие другой. Отдельно подтверждено практикой, что «конфиденциальные данные» – это не декларация в описании поля, а измеримое свойство: утверждение о шифровании было проверено не чтением кода, а прямым поиском секретного значения в бинарном файле базы данных, и только этот эксперимент, а не сам факт вызова функции **Encrypt**, можно считать доказательством защищённости.

С прагматической точки зрения работа показала, что большая часть типовых угроз из теоретической части задания (перебор, разглашение данных при компрометации канала или носителя, использование уязвимостей в коде) закрывается не экзотическими средствами, а корректным применением стандартных, хорошо документированных механизмов платформы – PBKDF2, AES-GCM, JWT с флагами cookie, именованный CORS, параметризованные запросы Entity Framework Core – при условии, что каждый из них применён осознанно и с понятным обоснованием, а не скопирован по шаблону. Также показано, что автоматизированная проверка защищённости (скрипт **verify.sh**, повторяющий сценарий атаки или обхода для каждого рубежа защиты и утверждающий код ответа и содержимое файла базы) даёт существенно более убедительное обоснование защищённости, чем ссылка на код или документацию, и может быть воспроизведена при последующих изменениях приложения для предотвращения регрессий безопасности.